

# TenantFlow Architecture Report

v1.0.0 — Automated Assurance, Zero-Regression & Container Orchestration

DOCUMENT ID: TF-ARCH-BP-100

AUTHOR: @EagleSoft461

STATUS: DRAFT / IN PROGRESS

TARGET RELEASE: v1.0.0 (Production-Ready)

CLASSIFICATION: INTERNAL / ENGINEERING  
CONFIDENTIAL

PROJECT: TenantFlow B2B SaaS Framework

## 1. EXECUTIVE SUMMARY

This final blueprint outlines the roadmap for the v1.0.0 Production-Ready release of the TenantFlow framework. Following the successful hardening of the global exception pipeline and perimeter security in v0.9.0, the current phase focuses entirely on system reliability and deployment mobility. By implementing strict automated assurance pipelines and containerizing the infrastructure, TenantFlow will achieve zero-regression guarantees and environment-agnostic deployment capabilities.

## 2. CORE IMPLEMENTATION STRATEGY

### Phase 1: Automated Assurance (Zero-Regression Pipeline)

- Unit Testing (JUnit 5 & Mockito):** Isolate and validate core business logic. Focus on verifying `TenantIdentifierResolver`, `GlobalExceptionHandler`, and service-layer algorithms without database dependency.
- Integration Testing (Testcontainers & Spring Boot Test):** Launch ephemeral Dockerized PostgreSQL instances during the test lifecycle to validate actual database interactions, dynamic schema provisioning, and JDBC routing accuracy.
- API Contract Testing:** Utilize `MockMvc` to assert that REST endpoints return strictly formatted RFC-7807 JSON payloads and correct HTTP status codes under failure conditions.

### Phase 2: Container Orchestration (Deployment Mobility)

- Application Containerization:** Draft a multi-stage Dockerfile leveraging Java 17 to build and run the Spring Boot artifact securely.
- Topology Orchestration:** Develop a `docker-compose.yml` file to synchronize the application container with a dedicated PostgreSQL 15 database instance, configuring correct network bindings, volume mounts for data persistence, and health checks.

### 3. FINAL DELIVERABLE (V1.0.0)

Upon completion, the TenantFlow framework will be fully deployable via a single `docker-compose up` command, fortified by an automated test suite that prevents future regressions. This marks the transition from a development framework into an enterprise-grade, production-ready SaaS engine.